4

The GCC Compiler

- One of the most popular C compiler
- Supplied with Linux, but available for many other platforms

In the command line: gcc -o hello hello.c

5

Integrated Development Environments

- Alternative to ***command line*** compiler
- Supplied with Linux, but available for many other platforms
- A software package that allows:
  - Edit
  - Compile
  - Link
  - Execute
  - Debug
  - Syntax Highlighting

6

Integrated Development Environments

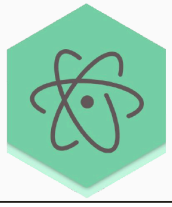
Popular IDEs are:











10

General form of a Simple Program
Functions

- Named block of executable code
- Building blocks from which programs are constructed
- Concept came from mathematics
  - f(x) -> x+1
- Functions which computes values uses *return* statement to specify what value it **returns**
- Two categories:
  - User defined functions
  - Provided as a part of C implementation -> Library functions

11

General form of a Simple Program
Functions

The *main* function:

- The *main* function is mandatory
- Gets automatically called when the program is executed
- Returns a status code; the value 0 means normal program execution
- Many compilers will produce warning messages if no *return* statement is provided at the end of *main* function

12

General form of a Simple Program
Statements

- A ***statement*** is a command to be executed when the program runs
  - Example from *hello.c*:
  - printf("Hello, world\n");
- Our previous program ***hello.c*** uses only two types of statements:
  - ***return*** statement
  - function call -> Asking a function to perform its assigned task
- C requires that each statement end with a semicolon
  - There's one exception: the compound statement

16

Comments
C99 Style

- ***comment*** begins with ***//***

```
// Name: hello.c
// Purpose: Prints Hello, world
// Author: Mohammed Saidul Islam

#include <stdio.h>
int main(void){   // beginning
      printf("Hello, world\n");   // print function
      return 0;
}
```

- Less error prone : no chance of unterminated comment to consume part of a program
- Simple and helpful because it ends automatically at the end of the line
- Multiline comments stands out better

17

Variables and Assignment
Types

- Most programs need a way to store data temporarily
- These storage locations are called ***variables***
- Each variable has a ***type*** -> specifies the type of data it will store
- Example:
  - int
    - Short for *integer*, E.g.: 0, -243, 243
    - Can store whole numbers
    - Limited range

18

Variables and Assignment
Types

- Example:
  - float
    - Short for *floating point* numbers
    - Can store much larger numbers than *int*
    - Can stores numbers with digits after the decimal point, like 379.125
    - Limitations are:
      - Slower arithmetic operations
      - Approximate nature of *floating point* values (i.e., round-off error)

22

Variables and Assignment

Printing the value of a variable

- *printf* can be used to display the current value of a variable
- To write the message
  - Height: *h*

    where *h* is the current value of the height variable, we'd us the following call of *printf*

    printf("Height: %d\n", height);
  - *%d* is a placeholder indicating where the value of *height* is to be filled in
  - *%d* works for *int*, *%f* is required for *float*

23

Variables and Assignment

Printing the value of a variable

- By default, *%f* displays a number with six digits after the decimal points
- To force *%f* to display *p* digits after the decimal point, put *.p* between % and f
- To print the line
  - Profit: $2150.48

    We have to use the following printf:

    printf("Profit: $%.2f\n", profit);

24

Variables and Assignment

Printing the value of a variable

- There's no limit to the number of variables that can be printed by a single call of *printf*:
  - printf("Height: %d Length: %d\n", height, length);

28

Reading Input

- scanf is the C library's counterpart to printf
- scanf requires a ***format string*** to specify the appearance of the input data
- Example of using scanf to read an int value:
  - scanf("%d", &i) //reads an integer and stores it into *i*
- To read a float value:
  - scanf("%f", &x) //reads a float and stores it into *x*
- The & symbol is usually (but not always) required when using scanf

29

Program (Revisited)

Computing the dimensional weight of a box

- We can modify our previous program to take inputs from user using scanf
- Each call of scanf is immediately preceded by a call of printf that displays a ***prompt.***
- Sample output of the program:

```
Enter height of the box: 8
Enter length of the box: 12
Enter width of the box: 10
Volume (cubic inches): 960
Dimensional weight (pounds): 6
```

30

Defining Names for constants

- Both of the programs rely on the constant 166, whose meaning may not be clear to someone reading the program.
- Using a feature known as ***macro definition,*** we can name this constant:

```
#define INCHES_PER_POUND 166
```

- When a program is compiled, the preprocessor replaces each macro by the value that it represents

34

Identifiers

- Names for variables, functions, macros, and other entities are called ***identifiers***.
- An identifier may contain letters, digits, and underscores, but must begin with a letter or underscore:

  times10 get_next_char _done

  It's usually best to avoid identifiers that begin with an underscore.
- Examples of illegal identifiers:

  10times get-next-char

35

Identifiers

- C is ***case-sensitive:*** it distinguishes between upper-case and lower-case letters in identifiers
- For example, the following identifiers are all different:
  - job joB jOb jOB Job JoB JOb JOB
- Many programmers use only lower-case letters in identifiers (other than macros), with underscores inserted for legibility:

  symbol_table current_page name_and_address

36

Identifiers

- Other programmers use an upper-case letter to begin each word within an identifier:

  symbolTable currentPage nameAndAddress
- C places no limit on the maximum length of an identifier